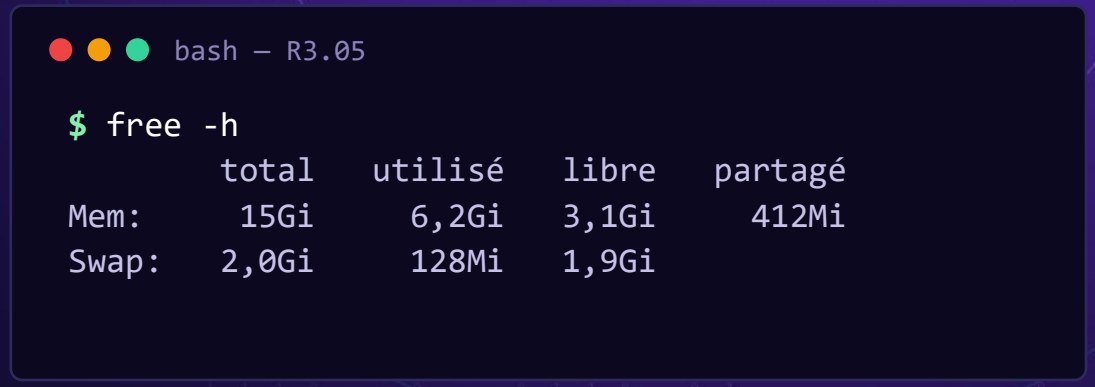
lamae

BUT2 INFORMATIQUE · RESSOURCE R3.05

Programmation système

CM 07b — La mémoire virtuelle (2/2)



Christophe Cruz · christophe.cruz@ube.fr · 2026-2027

IUT Dijon-Auxerre · Université Bourgogne Europe

PLAN

Au programme

01

La pagination

Pages et cadres : un espace contigu, plus grand que la mémoire, et protégé.

02

La traduction d'adresse

Numéro de page + décalage, table des pages, tables à plusieurs niveaux, TLB.

03

Les défauts de page

Charger à la demande, choisir une victime : FIFO, LRU, optimal.

04

Le swap sous Linux

La zone d'échange, swappiness, observer la mémoire.

Prérequis : CM 07a (MMU, adresses virtuelles et physiques, fragmentation).

01

La pagination

Découper la mémoire en morceaux de taille fixe

Pourquoi paginer ?

01

Un espace contigu

Chaque processus voit un grand espace d'adresses continu, même si sa mémoire physique est éparpillée : la fragmentation externe disparaît.

02

Plus grand que la RAM

L'espace virtuel peut dépasser la mémoire physique : les pages peu utilisées attendent sur le disque. Notre programme de 12 Mo tient dans 10 Mo.

03

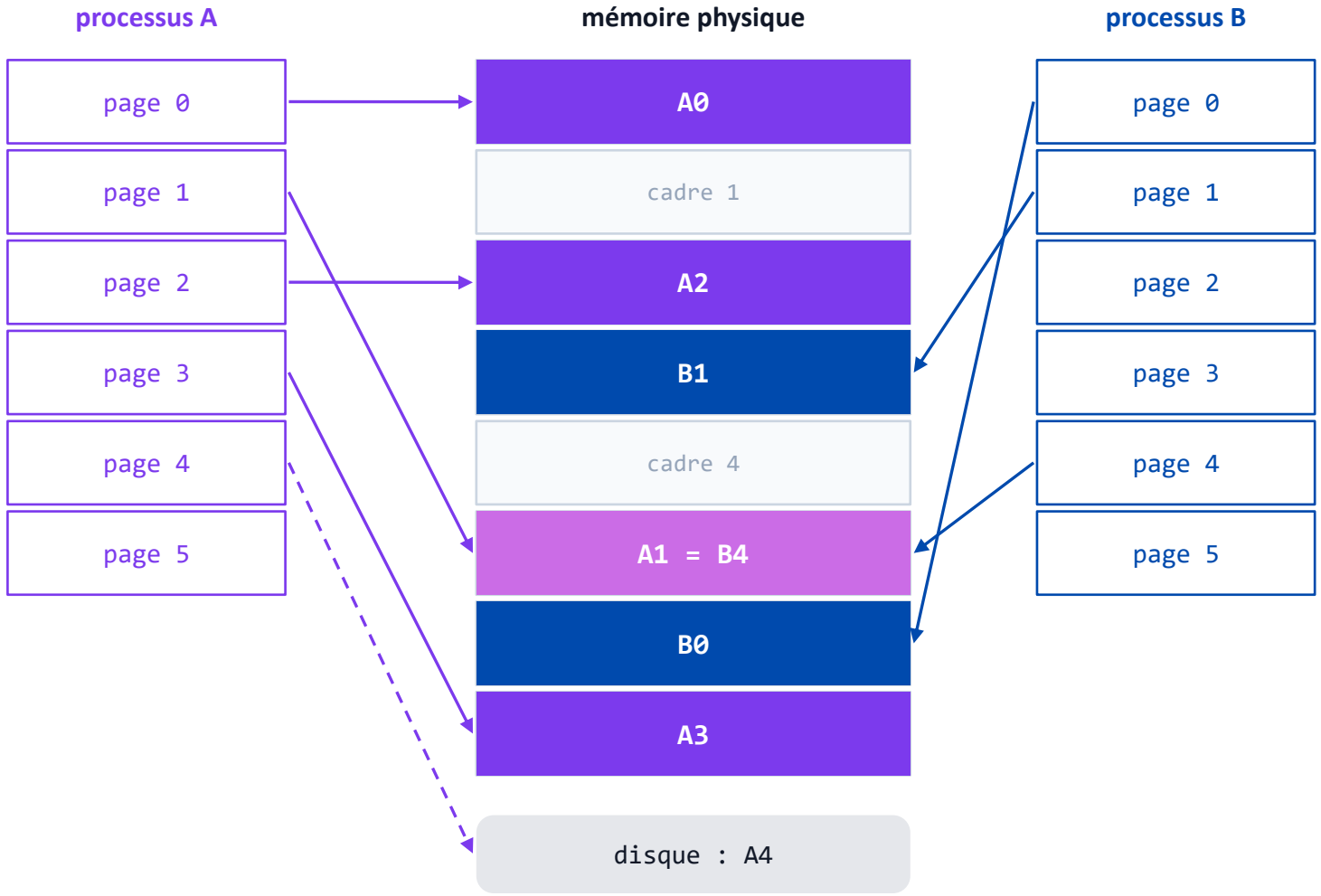
Protection et partage

Chaque page a des droits (lecture, écriture, exécution). Deux processus peuvent partager une page (code, bibliothèques, copie à l'écriture).

Utilisée par quasiment toutes les architectures modernes : x86, ARM, RISC-V (pas toujours dans l'embarqué).

Pages et cadres

- L'espace virtuel est découpé en pages, la mémoire physique en cadres (frames) de même taille
- Taille typique : 4 Kio
- N'importe quelle page peut aller dans n'importe quel cadre
- Une page peut aussi être sur le disque



02

La traduction d'adresse

Ce que fait la MMU à chaque accès mémoire

Une adresse = page + décalage

Avec des adresses de 32 bits et des pages de 4 Kio = 2^{12} octets :




```
0x00403A7C  →  page 0x00403 , décalage 0xA7C
```

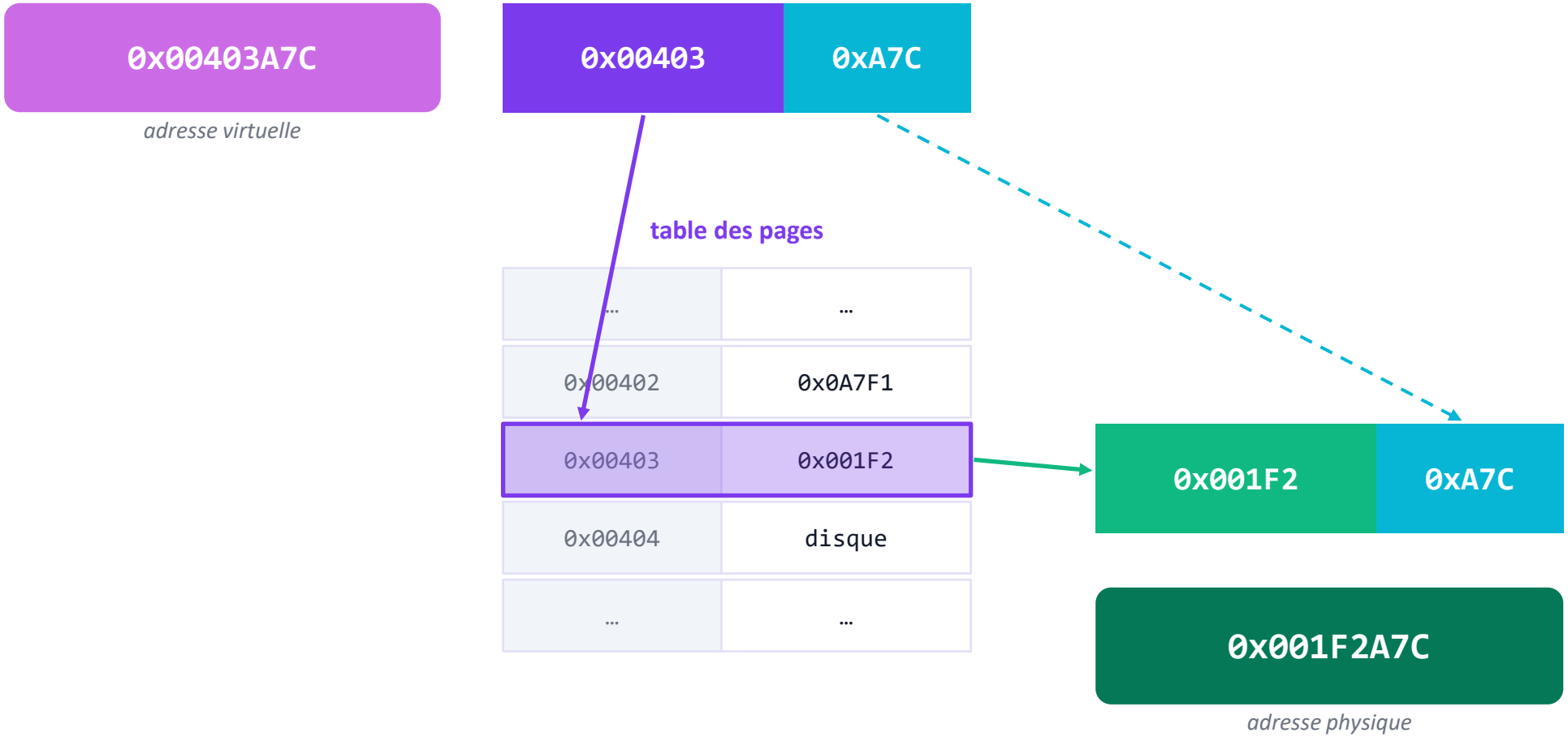
Le décalage ne change pas

Il désigne l'octet dans la page ; c'est le même dans le cadre.

Seul le numéro est traduit

page → cadre, grâce à la table des pages du processus.

La traduction pas à pas



Chaque processus a sa propre table : le noyau change de table à chaque changement de contexte (registre CR3 sur x86).

Une entrée de la table des pages

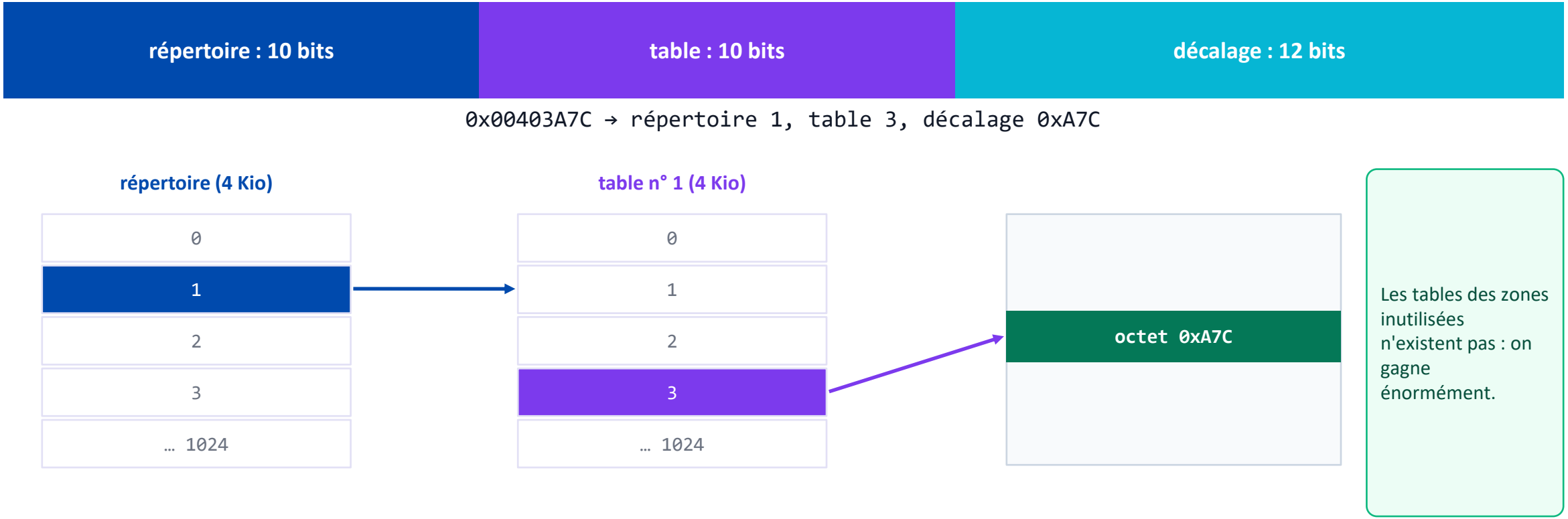


entrée de 32 bits (x86) : 20 bits pour le cadre, 12 bits de drapeaux

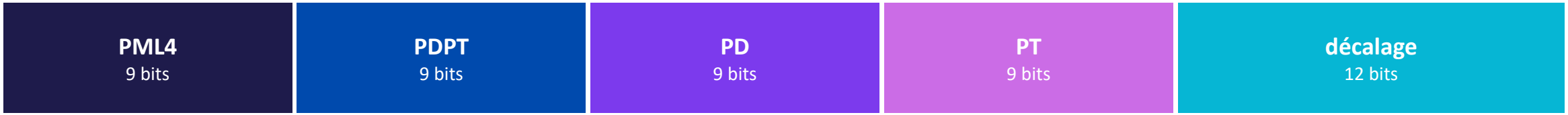
Bit	Nom	Rôle
P	présent	1 : la page est en mémoire ; 0 : accès → défaut de page
R/W	lecture / écriture	0 : page en lecture seule (code, copie à l'écriture)
U/S	utilisateur / superviseur	0 : réservée au noyau
A	accédée	mise à 1 par la MMU à chaque accès (utile pour LRU)
D	modifiée (dirty)	mise à 1 à chaque écriture : il faudra la recopier sur le disque
NX	non exécutable	(x86-64) interdit d'exécuter du code dans la page : la pile, le tas

Une table trop grande : deux niveaux

Problème : 2^{20} entrées × 4 octets = 4 Mo de table par processus, même pour un programme minuscule.



En 64 bits : 4 ou 5 niveaux



adresse virtuelle de 48 bits utiles : 256 Tio adressables

4 niveaux

Utilisés par Linux sur x86-64 depuis le noyau 2.6.11. Chaque table fait 4 Kio et contient 512 entrées de 8 octets.

5 niveaux

Depuis le noyau 4.14 (processeurs récents) : adresses de 57 bits, soit 128 Pio. Pour les très gros serveurs.

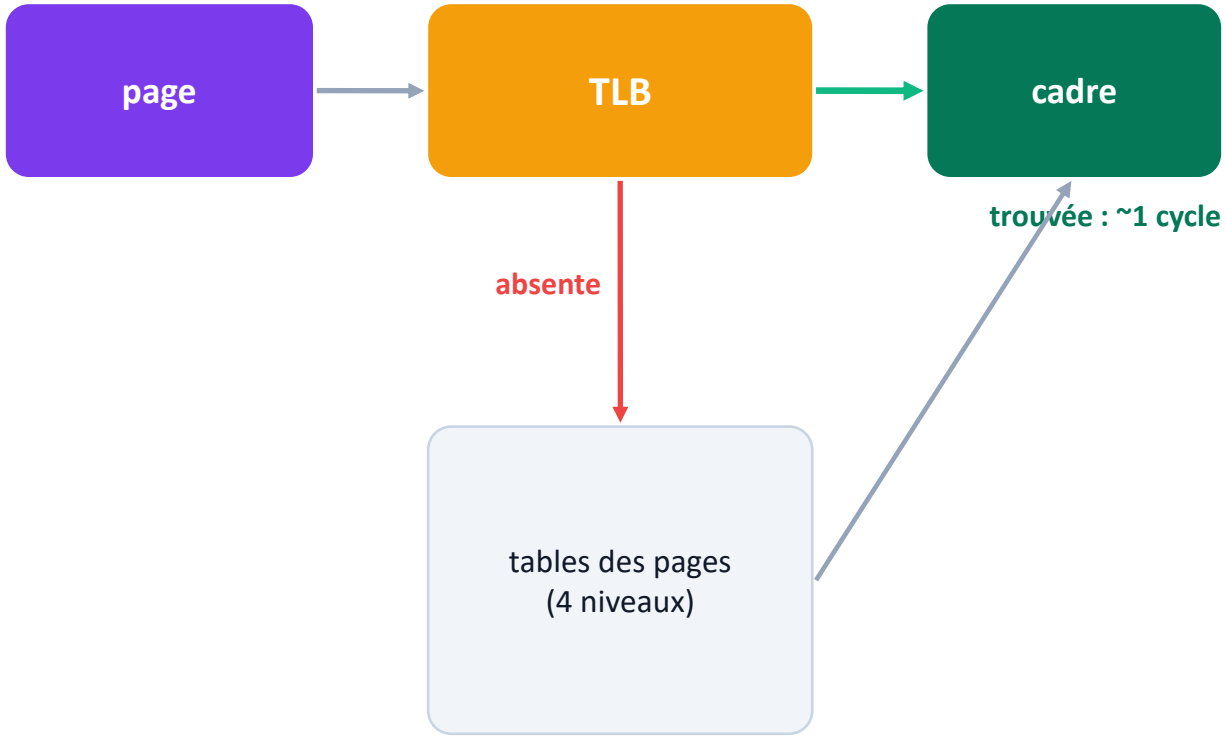
Grandes pages

On peut arrêter la descente plus tôt : pages de 2 Mio ou 1 Gio (huge pages). Moins d'entrées, moins de défauts.

Coût : sans aide, chaque accès mémoire en demanderait 4 ou 5 de plus pour parcourir les tables. D'où la TLB.

Le cache de traduction : la TLB

- La TLB (Translation Lookaside Buffer) garde les dernières traductions page → cadre
- Elle est petite (quelques centaines à quelques milliers d'entrées) mais très rapide
- Grâce à la localité des programmes, plus de 99 % des accès la trouvent
- Changer de processus vide la TLB (ou l'étiquette) : c'est une partie du coût d'un changement de contexte



parcours : plusieurs accès mémoire, puis mise à jour de la TLB

03

Les défauts de page

Quand la page demandée n'est pas en mémoire

Trois cas à chaque traduction

01

Entrée valide, page en mémoire

La MMU remplace le numéro de page par le numéro de cadre : l'accès se fait. C'est le cas normal.

02

Page jamais encore utilisée

Défaut de page « mineur » : le noyau trouve un cadre libre, le met à zéro, remplit l'entrée, et l'instruction reprend. C'est l'allocation paresseuse de malloc. Si l'adresse n'appartient à aucune zone du processus : SIGSEGV.

03

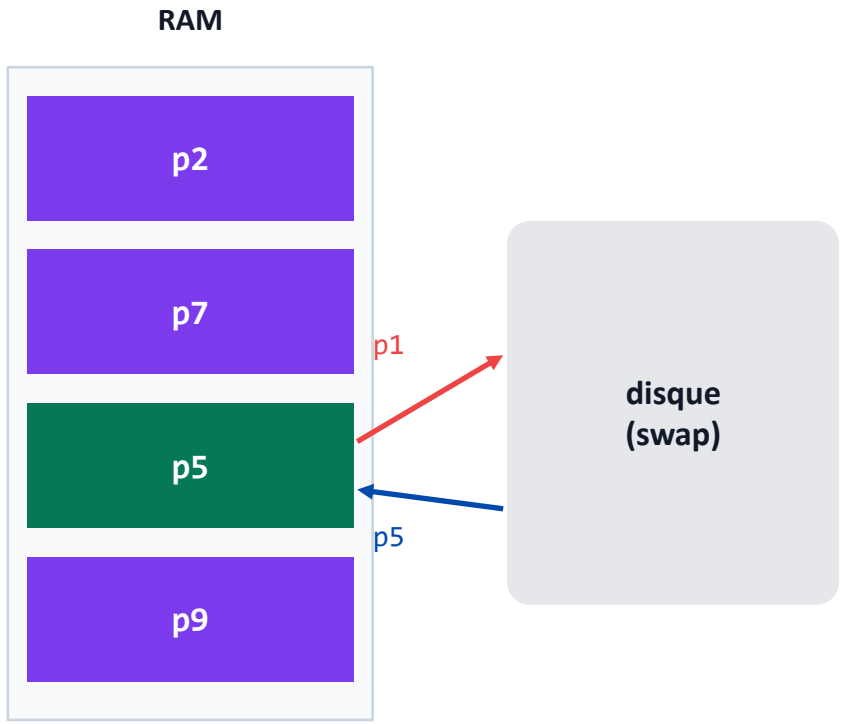
Page sur le disque (swap)

Défaut de page « majeur » : il faut lire la page sur le disque et la placer dans un cadre. Des milliers de fois plus lent qu'un accès en mémoire.

```
$ /usr/bin/time -v ./programme 2>&1 | grep faults
        Major (requiring I/O) page faults: 0
        Minor (reclaiming a frame) page faults: 1532
```

Un défaut de page, pas à pas

1. Le processus accède à la page 5 : bit P = 0
2. La MMU déclenche une exception : on passe dans le noyau
3. Plus de cadre libre : le noyau choisit une page victime
4. Si la victime est modifiée (D = 1), elle est écrite sur le disque
5. La page 5 est lue depuis le disque dans le cadre libéré
6. La table est mise à jour (P = 1), l'instruction est relancée



Le processus ne voit rien : il est simplement endormi pendant la lecture du disque (état D).

Quelle page sacrifier ?

Optimal (Belady)

Remplacer la page qui servira le plus tard. Impossible à réaliser (il faudrait connaître l'avenir), mais sert de référence.

FIFO

Remplacer la page chargée depuis le plus longtemps. Simple, mais elle peut jeter une page très utilisée.

LRU

Least Recently Used : remplacer la page inutilisée depuis le plus longtemps. Très bon, mais coûteux à suivre exactement.

Horloge (seconde chance)

FIFO qui épargne les pages dont le bit A vaut 1 (en le remettant à 0). Une bonne approximation de LRU, utilisée en pratique.

Exercice : FIFO avec 3 cadres

références	7	0	1	2	0	3	0	4	2	3
cadre 1	7	7	7	2	2	2	2	4	4	4
cadre 2		0	0	0	0	3	3	3	2	2
cadre 3			1	1	1	1	0	0	0	3
défaut ?	oui	oui	oui	oui	non	oui	oui	oui	oui	oui

FIFO : 9 défauts

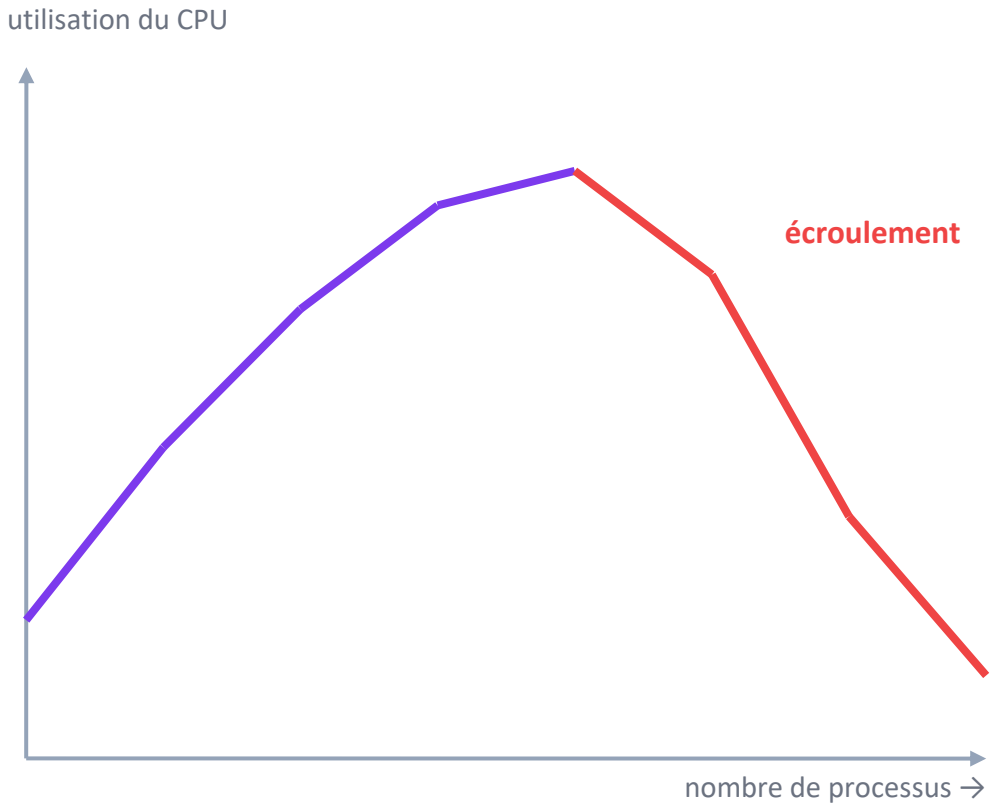
LRU : 8 défauts

Optimal : 6 défauts

Refaites l'exercice avec LRU pour vérifier : on remplace la page utilisée le moins récemment.

L'écroulement (thrashing)

- Un processus a besoin d'un certain nombre de pages en même temps : son ensemble de travail (working set)
- Si la mémoire ne peut pas contenir les ensembles de travail de tous les processus, chaque accès provoque un défaut de page
- Le système passe son temps à échanger des pages : le processeur est inactif, le disque saturé, la machine semble gelée



Remèdes : ajouter de la RAM, lancer moins de processus à la fois, ou laisser le noyau tuer un processus gourmand (OOM killer).

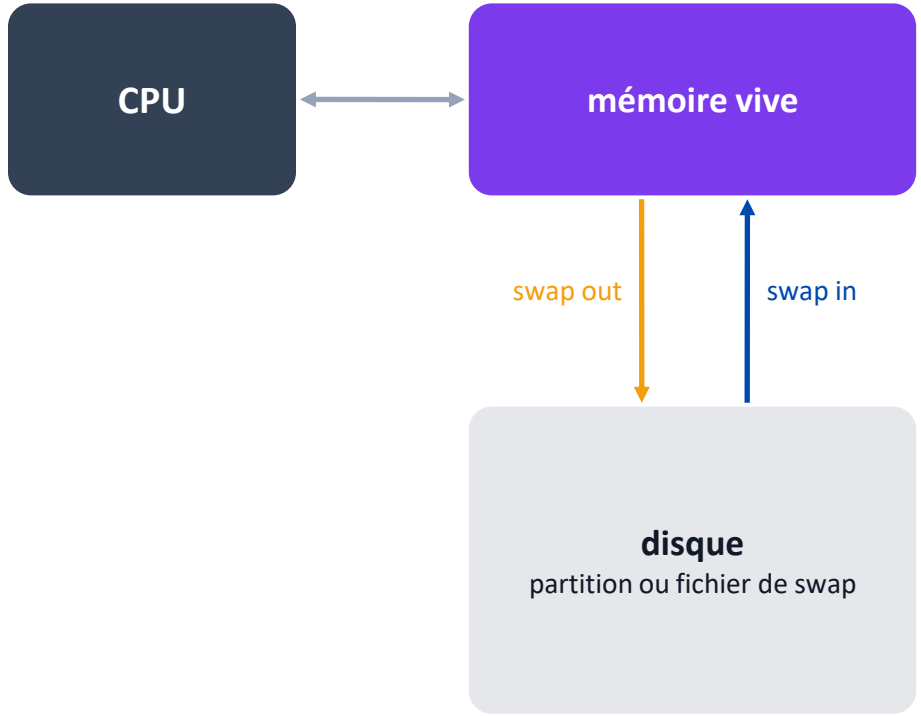
04

Le swap sous Linux

La zone d'échange et son réglage

La zone d'échange (swap)

- L'espace virtuel dépasse la mémoire physique : Linux utilise une zone d'échange sur le disque, une partition ou un fichier
- Ce n'est pas un cache : une page swappée n'est plus en mémoire
- Le swap doit servir le moins possible : un échange trop fréquent ralentit tout le système
- Taille : longtemps « 1 à 1,5 fois la RAM » ; aujourd'hui souvent quelques Go, ou au moins la RAM pour l'hibernation




```
$ swapon --show
NAME      TYPE SIZE  USED PRIO
/swapfile file   2G  128M   -2
```

Régler le swap : swappiness

Le paramètre /proc/sys/vm/swappiness indique à Linux s'il doit plutôt libérer le cache des fichiers ou plutôt swapper les pages des processus.



0 : swapper le moins possible **60 : défaut** 100 : coût égal fichiers / swap 200 : swapper volontiers

terminal

```
$ cat /proc/sys/vm/swappiness
60
$ sudo sysctl vm.swappiness=10        # jusqu'au redémarrage
vm.swappiness = 10
$ echo "vm.swappiness=10" | sudo tee /etc/sysctl.d/99-swap.conf   # permanent
```

Mise à jour : la valeur va de 0 à 200 depuis le noyau 5.8 (de 0 à 100 avant). Une valeur basse convient aux postes de travail bien dotés en RAM.

Observer la mémoire

free

```
$ free -h
               total        utilisé      libre      tampon/cache    disponible
Mem:            15Gi         6,2Gi      3,1Gi             6,0Gi          8,9Gi
Swap:          2,0Gi         128Mi      1,9Gi
```

vmstat : si / so = swap in / out

```
$ vmstat 1
 procs -----memory----- ---swap-- ----io----
 r  b   free  cache     si   so    bi   bo
 1  0  3.1G   6.0G      0    0     4   20
```

```
$ grep -E "VmSize|VmRSS|VmSwap" /proc/3121/status
VmSize:   245760 kB     # mémoire virtuelle réservée
VmRSS:     41232 kB     # réellement en RAM
VmSwap:        0 kB     # sur le disque
```

« libre » n'est pas « disponible »

Linux remplit la RAM inutilisée avec le cache des fichiers, qu'il libère au besoin.

si et so

Des valeurs durablement non nulles : la machine manque de mémoire et swappe.

VmSize ≫ VmRSS

Normal : allocation paresseuse et pages partagées.

Linux, en résumé

Segmentation

Presque absente : modèle plat, segments utilisés seulement pour les privilèges (anneaux 0 et 3).

Pagination

4 niveaux sur x86-64 (5 sur les processeurs récents), pages de 4 Kio et grandes pages de 2 Mio ou 1 Gio.

Allocation paresseuse

malloc ne réserve que de l'espace virtuel ; les cadres sont attribués au premier accès (défaut mineur).

Copie à l'écriture

Après fork, père et fils partagent les pages en lecture seule ; une page n'est copiée que si l'un écrit (CM 03a).

Surallocation

Le noyau promet plus de mémoire qu'il n'en a ; s'il en manque vraiment, l'OOM killer tue un processus (CM 01b).

Swap

Zone d'échange sur disque, réglée par swappiness, observable avec free et vmstat.

SYNTHÈSE

À retenir

01

Pages et cadres

Même taille (4 Kio). N'importe quelle page dans n'importe quel cadre, ou sur le disque.

02

Traduction

Adresse = n° de page + décalage. La table des pages donne le cadre ; le décalage ne change pas.

03

Tables à niveaux

2 niveaux en 32 bits, 4 ou 5 en 64 bits : seules les tables utiles existent. La TLB accélère tout.

04

Défaut de page

Mineur (page neuve) ou majeur (page sur le disque). Le noyau choisit une victime.

05

Remplacement

Optimal (théorique), FIFO, LRU, horloge. Exercice : 9, 8 et 6 défauts.

06

Swap

Utile mais lent ; trop de swap mène à l'écroulement. Réglage : swappiness.

Ressources pédagogiques

Explorer sur votre machine

```
$ getconf PAGE_SIZE
4096
$ cat /proc/meminfo
$ cat /proc/sys/vm/swappiness
$ /usr/bin/time -v ./prog
$ man 5 proc
```

Écrivez un programme qui alloue 1 Go avec malloc sans y toucher, puis en écrivant dedans : comparez VmSize et VmRSS.

Lectures

- A. Tanenbaum, Systèmes d'exploitation : « Mémoire virtuelle » et « Algorithmes de remplacement de pages »
- R. et A. Arpaci-Dusseau, Operating Systems: Three Easy Pieces : chapitres 18 à 23 (pagination, TLB, swap)
- M. Gorman, Understanding the Linux Virtual Memory Manager (gratuit en ligne)

lamae

Questions ?

PROCHAINE SÉANCE

TD / TP — Mise en pratique

threads et mutex · sockets · projets sur machine

Christophe Cruz · christophe.cruz@ube.fr